

Övning 10: NP-fullständiga problem

DD2350 Algoritmer, datastrukturer och komplexitet

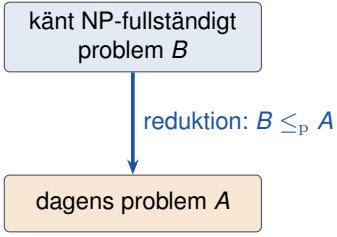
Marcus Dicander
EECS / Teoretisk datalogi, KTH

HT 2026

Idag

- **Dagens fem uppgifter** – en bild var, så att ni hinner börja fundera.
- **Verktygslådan:** reduktioner åt rätt håll, receptet för NP-fullständighetsbevis och varför indata mäts i bitar.
- **Lösningar:**
  - Uppgift 1: Konstruera kappsäckslösning (en turingreduktion)
  - Uppgift 2: Tillförlitlighet hos Internet
  - Uppgift 3: Håstads leksaksaffär
  - Uppgift 4: Processorschemaläggning
  - Uppgift 5: Lapptäcke
- Sammanfattning, vanliga fel och nästa gång.

Allt som är märkt **Kaninhål** är frivillig fördjupning. Det är å andra sidan där det roliga finns.



Vore A lätt så vore B lätt.
B är inte lätt. Alltså.

Dagens uppgifter

Uppgift 1: Konstruera kappsäckslösning

Kappsäcksproblemet (välkänt NP-fullständigt, föreläsning 21):

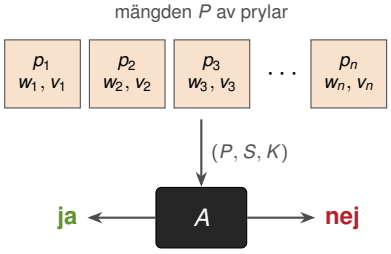
- **Indata:** en mängd P av prylar med vikt w_i och värde v_i, en kappsäcksstorlek S och ett mål K. Alla tal är ickenegativa heltal.
- **Fråga:** går det att välja ut prylar av sammanlagt värde minst K så att deras sammanlagda vikt är högst S?

Anta att vi har en algoritm A som löser detta beslutsproblem.

Uppgift

Konstruera en algoritm som med samma indata som A dels besvarar kappsäcksproblemet, dels – om svaret är ja – talar om precis *vilka* prylar som ska packas. Den får anropa A $O(|P|)$ gånger men får i övrigt inte ta mer än polynomisk tid.

Du konstruerar och analyserar alltså en *turingreduktion* av det konstruktiva kappsäcksproblemet till beslutsproblemet.

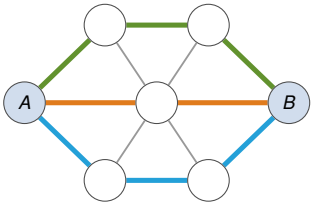


A svarar bara ja eller nej.
Vi vill veta *vilka* prylar.

Uppgift 2: Tillförlitlighet hos Internet

- Datorer och förbindelser går sönder. Man vill därför ha alternativa vägar mellan varje par av datorer.
- Finns tre vägar från A till B som är helt disjunkta (utom ändpunkterna) kan två datorer, vilka som helst, försvinna utan att A och B tappar kontakten.
- **Modell:** oriktad graf; hörn = datorer, kant (X, Y) = *möjlig* direktförbindelse.
- Tillförlitlighet $T(X, Y)$: antalet disjunkta vägar som ska finnas mellan X och Y.
- Budget B: antalet direktförbindelser man har råd att bygga.

grått: övriga möjliga förbindelser



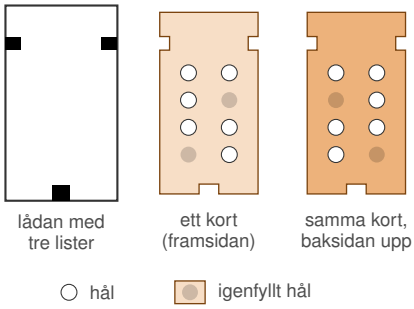
Tre disjunkta vägar mellan
A och B: två datorer kan försvinna.

Uppgift

Går det att plocka ut en delgraf bestående av B kanter så att det för alla par av hörn (X, Y) finns minst $T(X, Y)$ disjunkta stigar i delgrafen mellan X och Y? Visa att detta problem är NP-fullständigt!

Uppgift 3: Håstads leksaksaffär

- Håstads leksaksaffär säljer ett pussel som består av en låda och ett antal kort.
- Ett kort kan placeras i lådan på två sätt, med framsidan upp eller med baksidan upp, eftersom det finns urgröpningar i korten som måste passa in i lister som sitter i lådan.
- På varje kort finns två kolumner med hål, men vissa av hålen är igenfyllda.
- **Mål:** lägg korten i lådan så att hela botten täcks – för varje hålposition måste alltså åtminstone ett av korten ha ett igenfyllt hål.



Uppgift

Bevisa att språket $\textsc{Toys} = \{\langle c_1, c_2, \ldots, c_n\rangle : (c_i \text{ beskriver kort}) \wedge (\text{det går att lösa pusslet})\}$ är NP-fullständigt.

Uppgift 4: Processorschemaläggning

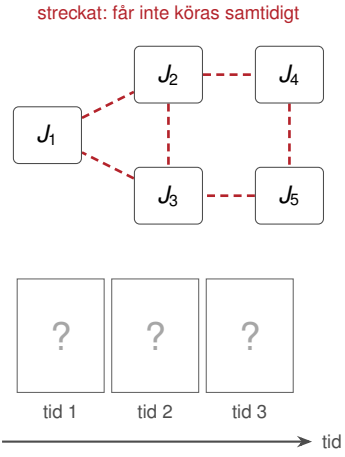
- Givet: n jobb som ska exekveras och ett tillräckligt antal processorer.
- Varje jobb tar en tidsenhet att utföra.
- Det finns också villkor på formen *jobb i kan inte exekveras samtidigt som jobb j*.
- SCHEDULE: går det att schemalägga jobben så att alla jobb kan exekveras på tre tidsenheter?

Uppgift

a) Visa att SCHEDULE är NP-fullständigt.

b) Är problemet fortfarande NP-fullständigt om vi kräver att jobben ska vara exekverade efter k tidsenheter där $k > 3$?

c) Är det sant för $k = 2$?

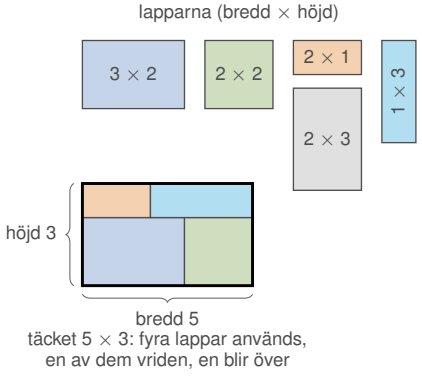


Uppgift 5: Lapptäcke

- *Patchwork*, från programvaruföretaget Stitcher, pusslar ihop rektangulära lappar till ett lapptäcke utan hål och utan lappar som överlappar varandra.
- **Indata:** en lista av rektangulära lappar (bredd och höjd i hela millimeter) och önskad storlek (bredd och höjd) på det färdiga rektangulära lapptäcket.
- **Utdata:** om en *delmängd* av lapparna kan pusslas ihop till ett täcke av rätt storlek – en beskrivning av hur. För varje lapp som används: var den ska placeras och om den behöver vridas.

Uppgift

Formulera detta konstruktionsoptimeringsproblem med matematisk notation och visa att det tillhörande beslutsproblemet (existerar en lösning?) är NP-fullständigt.



Fastnat? En ledtråd per uppgift

Fundera en stund innan vi löser dem. Den som vill ha hjälp på traven:

1. **Kappsäck:** fråga *A* vad som händer om en pryl försvinner.
2. **Internet:** reducera HAMILTONSK KRETS (Viggos egen ledning). Vilken budget och vilka T-värden tvingar fram en cykel?
3. **Håstad:** ett kort kan ligga på två sätt. Vad mer i den här kursen kan anta precis två värden?
4. **Schemaläggning:** rita jobben som hörn och konflikterna som kanter. Titta sedan länge på bilden.
5. **Lapptäcke:** gör täcket en millimeter högt.

Gemensamt för uppgift 2–5

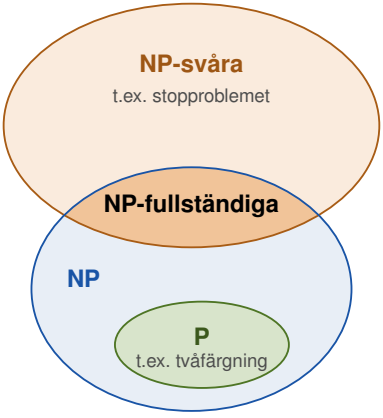
Hitta ett *känt* NP-fullständigt problem som liknar ditt, och visa att det kända problemet kan lösas med hjälp av ditt.

Det svåra är nästan aldrig beviset. Det svåra är att välja rätt kusin.

Verktygslådan

Repetition: NP, NP-svårt, NP-fullständigt

- **Beslutsproblem:** svaret är ja eller nej.
- **NP:** varje ja-instans har en lösning (ett *certifikat*) som kan verifieras på polynomisk tid.
- **NP-svårt:** *A* är NP-svårt om *varje* problem i NP kan reduceras till *A* på polynomisk tid.
- **NP-fullständigt:** $A \in \mathrm{NP}$ och *A* är NP-svårt.
- De NP-fullständiga problemen är alltså de svåraste i NP. Löser du ett av dem på polynomisk tid har du löst alla, och visat att $\mathrm{P} = \mathrm{NP}$.
- Vi behöver aldrig reducera *varje* problem i NP. Det räcker med *ett* som redan är NP-svårt, eftersom reduktioner kan kedjas.



NP-fullständiga: CNF-SAT, 3-färgning, hamiltoncykel, delmängdssumma, kappsäck, ...

(bilden förutsätter att $\mathrm{P} \neq \mathrm{NP}$)

Tre sorters problem

	Kappsäck som ...	**Svar**
Beslutsproblem	Finns en packning med värde $\geq K$ och vikt $\leq S$?	ja eller nej
Optimeringsproblem	Vilket är det största värde som ryms i säcken?	ett tal
Konstruktionsproblem	Vilka prylar ska packas?	en lösning

- NP och NP-fullständighet är definierade för **beslutsproblem**. Därför skriver vi om allt till ja/nej-frågor innan vi bevisar något.
- Den som kan konstruera en lösning kan förstås också svara ja eller nej. Åt det hållet är det gratis.
- **Uppgift 1** går åt andra hållet: från ja/nej-svar till en färdig packning.
- **Uppgift 5** ber oss först formulera ett konstruktionsproblem, och sedan bevisa att det tillhörande beslutsproblemet är NP-fullständigt.

Karpreduktion och turingreduktion

Karpreduktion av P till Q

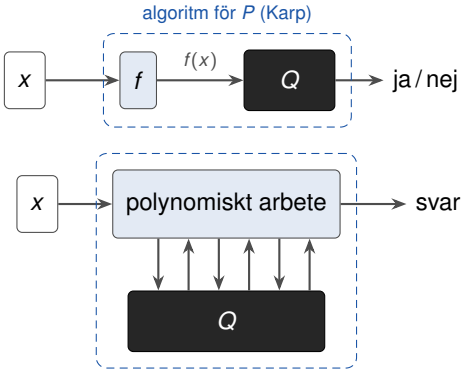
En algoritm som löser P med *exakt ett* anrop till Q, och där svaret från anropet returneras direkt. Allt annat arbete tar polynomisk tid.

Turingreduktion av P till Q

En algoritm som löser P genom att anropa Q en eller flera gånger. Allt annat arbete tar polynomisk tid.

Observera!

I ett NP-svårighetsbevis måste reduktionen vara en **karpreduktion**.



Kaninhål: varför duger inte en turingreduktion i ett NP-svårighetsbevis?

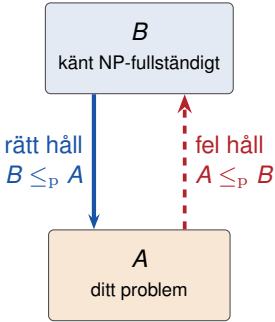
- Med en turingreduktion får man göra vad man vill med svaren. Till exempel negera dem.
- Då kan OSATISFIERBARHET (”saknar formeln lösning?”) reduceras till SAT: ett anrop, en negation, klart.
- Men ingen vet hur ett kort certifikat för att en formel *saknar* lösning skulle se ut. Det är frågan om $\mathrm{NP} = \text{co-NP}$, och de flesta tror att svaret är nej.
- Karpreduktioner bevarar däremot strukturen ”ja-instanser har certifikat”:

$$A \leq_{\mathrm{p}} B \text{ och } B \in \mathrm{NP} \implies A \in \mathrm{NP}.$$

  Certifikatet för $f(x)$ fungerar som certifikat för x.
- För konstruktions- och optimeringsproblem finns inget ja/nej-svar att skicka vidare. Där är turingreduktioner det naturliga verktyget – som i uppgift 1.

Reduktionens riktning

- Vi vill visa att *vårt* problem A är svårt.
- Då ska ett *känt* svårt problem B reduceras *till* A: $B \leq_{\mathrm{p}} A$.
- Läs $B \leq_{\mathrm{p}} A$ som ”B är inte svårare än A”. En snabb algoritm för A skulle ge en snabb algoritm för B.
- I den här kursen skriver vi reduktionen som en algoritm: *lös det kända problemet med hjälp av ditt eget.*
- **Fel håll** ($A \leq_{\mathrm{p}} B$) visar bara att A inte är svårare än B. Att reducera sitt problem till SAT bevisar mest att SAT-lösare är användbara.



Receptet: så bevisar man att A är NP-fullständigt

1. Visa att $A \in \mathrm{NP}$

a) Föreslå vad en lösning (ett certifikat) är.

b) Visa hur lösningen verifieras.

c) Visa att verifikationen tar polynomisk tid i indatas storlek.

2. Visa att A är NP-svårt

a) Välj ett känt NP-fullständigt problem B.

b) Beskriv en karpreduktion av B till A.

c) Visa att reduktionen tar polynomisk tid.

d) Visa att den är korrekt: ja-instanser blir ja-instanser och nej-instanser blir nej-instanser.

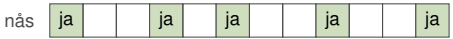
- Steg 2d är i praktiken två implikationer: ”x ja $\Rightarrow$ $f(x)$ ja” och ”$f(x)$ ja $\Rightarrow$ x ja”. Den andra är nej-till-nej skriven baklänges, och det är oftast där det intressanta händer.
- Alla fyra NP-fullständighetsbevisen i dag följer mallen till punkt och pricka. Det är en egenskap hos bevisen, inte en brist på fantasi.

Indatas storlek mäts i bitar

DELMÄNGDSSUMMA: har någon delmängd av heltalen $p_1, \ldots, p_n > 0$ summan exakt K?

```
function DelmängdssummaDP(p_1, ..., p_n, K)
    nås[0] ← ja;  nås[1..K] ← nej
    for i ← 1 to n do
        for s ← K downto p_i do
            if nås[s - p_i] then nås[s] ← ja
    return nås[K]
```

- Med enhetskostnad: $O(n \cdot K)$. Polynomiskt?
- K står i indata med $b \approx \log_2 K$ bitar. Uttryckt i b är tiden $O(n \cdot 2^b)$.
- En algoritm som är polynomisk i *talvärdena* i indata men exponentiell i indatas *längd* kallas **pseudopolynomisk**.



$\{3, 5, 8, 13\}$, $K = 11$: efter alla fyra talen. $3 + 8 = 11$.

K	bitar	rutor	vid 10^9 rutor/s
255	8	256	omärkligt
$\approx 10^6$	20	10^6	1 ms
$\approx 10^{18}$	60	10^{18}	32 år
$\approx 3 \cdot 10^{38}$	128	$3 \cdot 10^{38}$	10^{22} år

(tid för ett varv i den yttre slingan, alltså per tal p_i)

Kaninhål: går det att vara smartare? Inte såvitt någon vet. DELMÄNGDSSUMMA är NP-fullständigt: en polynomisk algoritm vore ett bevis för $\mathrm{P} = \mathrm{NP}$.

Fyra sätt att mäta samma algoritm

```
function ArithmeticSum(N)
    s ← 0
    for i ← 1 to N do
        s ← s + i
    return s
```

	enhetskostnad	bitkostnad
mätt i N	$O(N)$	$O(N \log N)$
mätt i $m = \log N$	$O(2^m)$	$O(m2^m)$

Pseudopolynomisk: ser bara polynomisk ut.

```
function SmartArithmeticSum(N)
    return N · (N+1) / 2
```

	enhetskostnad	bitkostnad
mätt i N	$O(1)$	$O(\log^2 N)$
mätt i $m = \log N$	$O(1)$	$O(m^2)$

Polynomisk på riktigt.

- Bitkostnad: addition av m-bitstal kostar $O(m)$, multiplikation och division räknar vi som $O(m^2)$.
- Det som avgör är den nedre raden: tiden som funktion av *antalet bitar m*.
- Pseudopolynomisk är en egenskap hos *algoritmen*, inte hos problemet. Summan gick att räkna ut smartare. För DELMÄNGDSSUMMA tror ingen att det går.

Kaninhål: varför just bitar?

- NP är definierad med turingmaskiner. Indata är en rad tecken på ett band.
- Maskinen vet inte att tecknen ”betyder” ett tal. Det enda den kan mäta sin tid mot är hur långt bandet är.
- Alltså: polynomisk tid betyder polynomisk i *antalet bitar* i indata.
- Den som vill fuska kan skriva talen unärt ($5 = 11111$). Då blir indata långt och allting ser polynomiskt ut. Därför är binär kodning underförstådd.
- **Konsekvens i dag:** verifikationer och reduktioner måste vara polynomiska i antalet bitar. En slinga som går S varv, eller en matris med $B \cdot H$ rutor, är inte det.



polynomisk tid: 8^c steg

pseudopolynomisk tid: 178^c steg

Vi kommer att möta bägge fallgroparna: kappsäck (uppgift 1) och lapptäcke (uppgift 5).

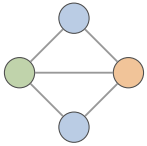
Dagens meny: kända NP-fullständiga problem

CNF-SAT

$(x_1 \vee \overline{x}_2)$
$\wedge (x_2 \vee x_3)$
$\wedge (\overline{x}_1 \vee \overline{x}_3)$

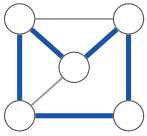
Kan variablerna ges värden så att varje klausul blir sann?

3-färgning



Kan hörnen färgas med tre färger så att grannar alltid får olika färg?

Hamiltoncykel



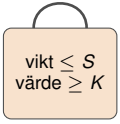
Finns en cykel som passerar varje hörn exakt en gång?

Delmängdssumma

$\{\mathbf{3}, 5, \mathbf{8}, 13\}$
$K = 11$

Finns en delmängd av de positiva heltalen $p_1, \ldots, p_n$ vars summa är exakt K?

Kappsäck



Finns prylar med sammanlagd vikt högst S och sammanlagt värde minst K?

- Alla fem är kända NP-fullständiga problem från föreläsningarna och får användas utan bevis.
- Fyra av dem blir utgångspunkt för var sitt bevis i dag. Den femte är redan huvudperson i uppgift 1.

Uppgift 1: Konstruera kappsäckslösning

Uppgift 1: Konstruera kappsäckslösning

Kappsäcksproblemet (välkänt NP-fullständigt, föreläsning 21):

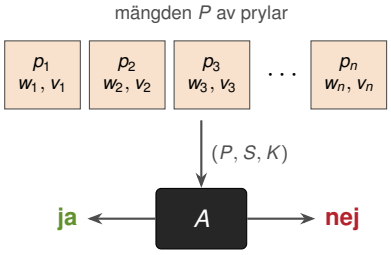
- **Indata:** en mängd P av prylar med vikt w_i och värde v_i, en kappsäcksstorlek S och ett mål K. Alla tal är ickenegativa heltal.
- **Fråga:** går det att välja ut prylar av sammanlagt värde minst K så att deras sammanlagda vikt är högst S?

Anta att vi har en algoritm A som löser detta beslutsproblem.

Uppgift

Konstruera en algoritm som med samma indata som A dels besvarar kappsäcksproblemet, dels – om svaret är ja – talar om precis *vilka* prylar som ska packas. Den får anropa A $O(|P|)$ gånger men får i övrigt inte ta mer än polynomisk tid.

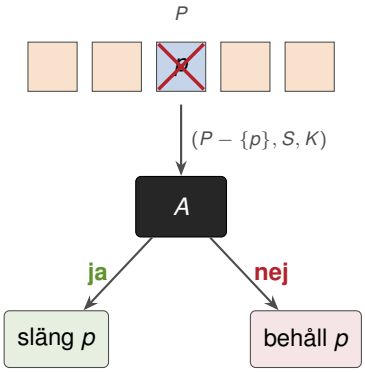
Du konstruerar och analyserar alltså en *turingreduktion* av det konstruktiva kappsäcksproblemet till beslutsproblemet.



A svarar bara ja eller nej.
Vi vill veta *vilka* prylar.

Idén: fråga om en pryl i taget

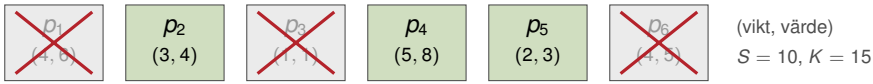
- $A(P, S, K)$ säger bara om det finns *någon* lösning. En instans kan ha noll, en eller flera.
- Fråga i stället A om en mindre instans: $A(P - \{p\}, S, K)$.
  - **ja**: det går att lösa problemet utan p. Släng p.
  - **nej**: det går *inte* att lösa problemet utan p. Behåll p.
- Gör så med varje pryl, en i taget. Det som är kvar när alla är prövade är en lösning.
- Samma idé med andra ord (Lisa Li, 2019): låtsas i tur och ordning att prylen p har oändligt stor vikt. Svarar A nej måste p vara med.



```
function Kappsäck(P, S, K)
    if not A(P, S, K) then return "Ingen lösning"
    foreach p in P do
        if A(P - {p}, S, K) then P ← P - {p}
    return P
```

- Slingan går över de *ursprungliga* prylarna, en gång var. Mängden P krymper under tiden.
- Antal anrop till A: ett före slingan och ett per pryl, alltså högst $|P| + 1 \in O(|P|)$.
- Allt övrigt arbete är mängdoperationer och tar polynomisk tid.
- Återstår att visa: mängden som returneras är verkligen en lösning – inte bara något som *innehåller* en lösning.

Exempel: sex prylar, $S = 10$, $K = 15$



0. $A(\{p_1, p_2, p_3, p_4, p_5, p_6\})$ = **ja**: det finns en lösning
1. $A(\{p_2, p_3, p_4, p_5, p_6\})$ = **ja**: släng p_1
2. $A(\{p_3, p_4, p_5, p_6\})$ = **nej**: behåll p_2
3. $A(\{p_2, p_4, p_5, p_6\})$ = **ja**: släng p_3
4. $A(\{p_2, p_5, p_6\})$ = **nej**: behåll p_4
5. $A(\{p_2, p_4, p_6\})$ = **nej**: behåll p_5
6. $A(\{p_2, p_4, p_5\})$ = **ja**: släng p_6

Resultat: $\{p_2, p_4, p_5\}$ med vikt $3 + 5 + 2 = 10 \leq S$ och värde $4 + 8 + 3 = 15 \geq K$.
Sju anrop till A, det vill säga $|P| + 1$.

Korrekthet, del 1: slingans invariant

Invariant

$A(P, S, K)$ = ja. Med andra ord: den aktuella mängden P innehåller minst en lösning.

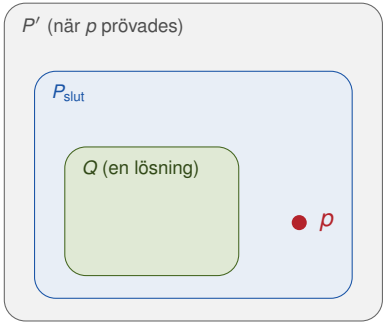
- **Före slingan:** första raden har just kontrollerat det. Annars hade vi redan returnerat ”Ingen lösning”.
- **Ett varv:** P ändras bara när $A(P - \{p\}, S, K)$ = ja, och då innehåller också det nya P en lösning.
- **Efter slingan:** mängden vi returnerar, P_{slut}, innehåller en lösning $Q \subseteq P_{\text{slut}}$.
- Det räcker inte riktigt. Vi lovade att P_{slut} *är* en lösning. Kan det ligga skräp kvar bredvid Q?
- Facit: nej, för slingan prövar varje pryl och plockar bort de överflödiga. Nästa bild skriver ut det argumentet i detalj.

Korrekthet, del 2: inget överflödigt blir kvar

Påstående: $Q = P_{\text{slut}}$.

- Anta motsatsen: det finns en pryl $p \in P_{\text{slut}}$ som inte ligger i Q.
- När p prövades var mängden $P' \supseteq P_{\text{slut}}$, för vi tar bara bort prylar.
- p fick vara kvar, så $A(P' - \{p\}, S, K)$ = nej.
- Men $Q \subseteq P_{\text{slut}} - \{p\} \subseteq P' - \{p\}$, och Q är en lösning. Svaret borde ha varit ja. Motsägelse.
- Alltså är P_{slut} själv en lösning: vikt högst S, värde minst K. □

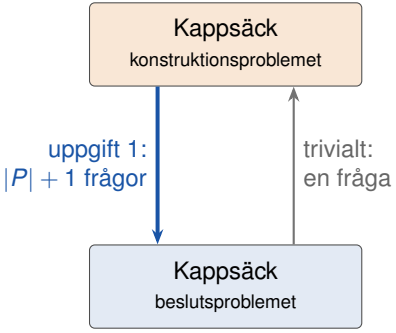
Kaninhål: varför räcker *ett* varv? För att mängden bara krymper. En pryl som var nödvändig när den prövades förblir nödvändig i varje mindre mängd.



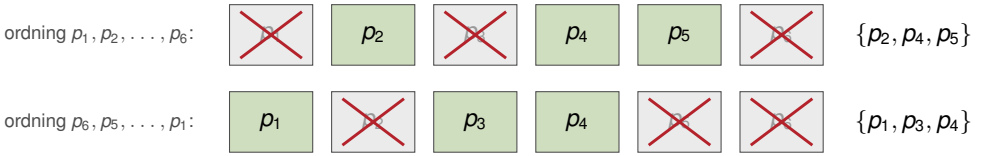
Q klarar sig utan *p*
– och det borde *A* ha sagt.

Analys: vad har vi visat?

- Högst $|P| + 1$ anrop till A, vart och ett på en instans som inte är större än den ursprungliga.
- Övrigt arbete: $O(|P|)$ mängdoperationer, polynomisk tid.
- Om beslutsproblemet går att lösa på tid $T(n)$ för indata av storlek n går konstruktionsproblemet alltså på tid $O(n \cdot T(n))$ plus polynomiskt småplock.
- **Slutsats:** att konstruera en packning är inte svårare än att avgöra om det finns någon – upp till en polynomisk faktor.
- Det här är en **turingreduktion**: flera anrop, efterarbete mellan anropen, och svaret är en mängd, inte ja eller nej.



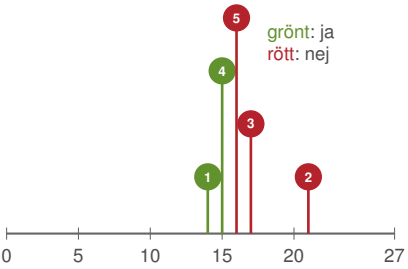
Kaninhål: spelar ordningen någon roll?



- För korrektheten: nej. För *vilken* lösning man får: ja. Båda har vikt 10 och värde 15.
- Exemplet har precis två lösningar. Algoritmen returnerar den som överlever ordningen.
- p_4 ingår i alla lösningar och behålls därför alltid. p_6 ingår inte i någon och slängs alltid.
- Uppgiften ber om ”precis vilka prylar som ska packas”. Den säger inte att packningen är unik – och det är den alltså inte.

Kaninhål: minimal är inte optimal

- Algoritmen lovar bara vikt $\leq S$ och värde $\geq K$. Sänk målet till $K = 12$ i exemplet, så returnerar den $\{p_4, p_6\}$: vikt 9, värde 13. Det bästa möjliga är 15.
- Vill man ha en *optimal* packning: leta först upp det största K som ger ja, och konstruera sedan.
- **Binärsökning** över $K \in [0, V]$ där $V = \sum v_i$: högst $\lceil \log_2(V + 1) \rceil$ anrop. log V är högst antalet bitar i indata, så det är polynomiskt många.
- **Linjärsökning** $K = V, V - 1, \ldots$ kan kräva V anrop. Det är pseudopolynomiskt: samma fälla som tabellen i verktygslådan.



$V = 27$: fem anrop ger $K^* = 15$

frågorna i ordning: 14, 21, 17, 15, 16

Dynamisk programmering, som i verktygslådan men med värden. Låt $M[i][c]$ vara det största värde som ryms i en säck med kapacitet c om bara prylarna $1, \ldots, i$ får användas:

$$M[i][c] = \begin{cases} 0 & \text{om } i = 0 \\ M[i-1][c] & \text{om } i > 0 \text{ och } c < w_i \\ \max\big(M[i-1][c],\ M[i-1][c-w_i] + v_i\big) & \text{annars} \end{cases}$$

- Svaret står i $M[|P|][S]$. Själva packningen får man genom att gå baklänges i tabellen.
- Tid $O(|P| \cdot S)$. $|P|$ är högst indatas längd, men S kan vara exponentiellt stort i antalet bitar. **Pseudopolynomiskt.**
- $S = 10^{18}$ ryms i 60 bitar. Tabellen får 10^{18} kolumner.

Ingen motsägelse

Kappsäck är NP-fullständigt *och* har en pseudopolynomisk algoritm. Sådana problem kallas *svagt* NP-fullständiga.

Algoritmen är utmärkt så länge talen är små. Det är de ofta i praktiken och sällan i bevis.

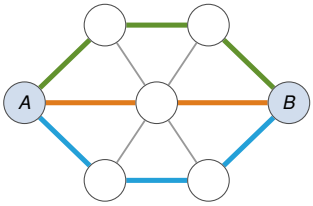
- **Hamiltoncykel:** ta bort en kant i taget. Behåll kanten om svaret blir nej. Kvar till slut: exakt en hamiltoncykel. Högst $|E| + 1$ frågor, samma bevis som nyss.
- **SAT:** sätt x_1 till sant och fråga om formeln fortfarande är satisfierbar. Om inte: sätt x_1 till falskt. Fortsätt med $x_2, \ldots, x_n$. Högst $n + 1$ frågor.
- **Klick, hörntäckning, delmängdssumma, . . . :** samma idé. Bygg lösningen en bit i taget och fråga efter varje steg om det går att fortsätta.
- **Allmänt:** för *varje* NP-fullständigt problem kan lösningar konstrueras med polynomiskt många frågor till beslutsproblemet. Frågan ”går den här början att fullborda till en lösning?” ligger nämligen själv i NP, och kan därför reduceras till problemet.
- Det är därför teorin kan nöja sig med ja/nej-frågor utan att något väsentligt går förlorat.

Uppgift 2: Tillförlitlighet hos Internet

Uppgift 2: Tillförlitlighet hos Internet

- Datorer och förbindelser går sönder. Man vill därför ha alternativa vägar mellan varje par av datorer.
- Finns tre vägar från A till B som är helt disjunkta (utom ändpunkterna) kan två datorer, vilka som helst, försvinna utan att A och B tappar kontakten.
- **Modell:** oriktad graf; hörn = datorer, kant (X, Y) = *möjlig* direktförbindelse.
- Tillförlitlighet $T(X, Y)$: antalet disjunkta vägar som ska finnas mellan X och Y.
- Budget B: antalet direktförbindelser man har råd att bygga.

grått: övriga möjliga förbindelser



Tre disjunkta vägar mellan A och B: två datorer kan försvinna.

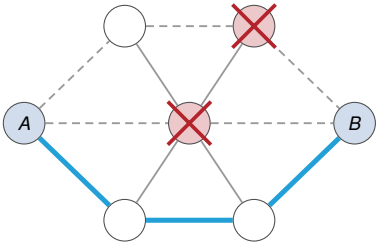
Uppgift

Går det att plocka ut en delgraf bestående av B kanter så att det för alla par av hörn (X, Y) finns minst $T(X, Y)$ disjunkta stigar i delgrafen mellan X och Y? Visa att detta problem är NP-fullständigt!

Vad betyder disjunkta stigar?

”We love internet. We hate when internet is down.”
(Lisa Li, 2019)

- Två stigar mellan X och Y är **disjunkta** om de är olika och inte har några gemensamma hörn utom ändpunkterna.
- Tre disjunkta stigar mellan A och B: slå ut två datorer, vilka som helst. Varje trasig dator förstör högst en stig, så minst en finns kvar.
- Det gäller också åt andra hållet (Mengers sats, 1927): om två icke-grannar inte kan skiljas åt genom att $k - 1$ andra hörn tas bort, så finns k disjunkta stigar mellan dem.
- $T(X, Y) = k$ betyder alltså: förbindelsen mellan X och Y ska tåla $k - 1$ trasiga datorer.



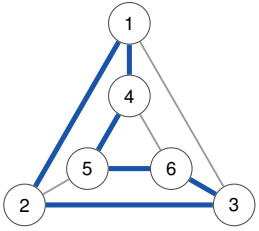
två datorer nere:
den blå stigen bär fortfarande

Steg 1: problemet ligger i NP

Indata: $G = (V, E)$, ett tal $T(X, Y)$ för varje par av hörn, och budgeten B.

- **Lösning (certifikat):** en kantmängd $E' \subseteq E$ och, för varje par (X, Y), en lista med $T(X, Y)$ stigar.
- **Verifiera:**
  a) att $G' = (V, E')$ är en delgraf till G,
  b) att G' har exakt B kanter,
  c) att stigarna för varje par är rätt antal, går i G' från X till Y och är parvis disjunkta.
- **Tid:** $O(|V|^2)$ par, och de disjunkta stigarna för ett par innehåller tillsammans högst $|V|$ hörn. Polynomiskt i indatas storlek.

Räcker inte E' ensam? Jo, men då ska verifieraren hitta stigarna själv. Det är ett **Kaninhål** längre fram.

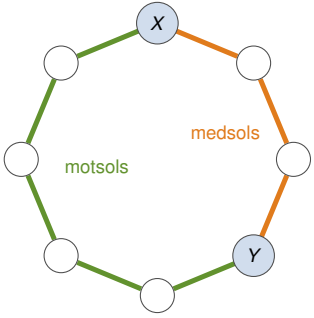


E': de sex blå kanterna

par	stigar
(2, 5)	2–3–6–5, 2–1–4–5
(1, 6)	1–2–3–6, 1–4–5–6
...	...

Steg 2: vilket känt problem liknar det här?

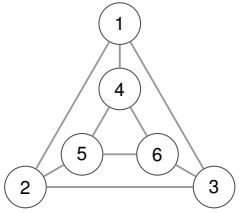
- Vilket är det billigaste sättet att ge *varje* par två disjunkta stigar? En ring.
- I en cykel finns det mellan två hörn precis två disjunkta stigar: medsols och motsols.
- En ring genom alla datorer, byggd av möjliga förbindelser, har ett namn. Den heter **hamiltoncykel**.
- HAMILTONCYKEL: givet en graf G, finns det en cykel som passerar varje hörn exakt en gång? Känt NP-fullständigt.
- **Plan:** kräv två stigar för alla par, och ge precis så många kanter som en ring behöver: $T \equiv 2$ och $B = |V|$.



Reduktionen

```
function Hamiltoncykel(G = (V, E))
    B ← |V|
    foreach par X, Y in V do
        T(X, Y) ← 2
    return InternetBeslut(G, T, B)
```

- Ett anrop, och svaret returneras direkt: en karpreduktion.
- Grafen skickas vidare oförändrad. T har $|V|(|V|-1)/2$ värden. Tid $O(|V|^2 + |E|)$, polynomiskt i indatas storlek.
- Reduktionen vet inte om G har en hamiltoncykel. Den behöver inte veta det. Den ska bara översätta frågan.



G: 6 hörn, 9 möjliga förbindelser

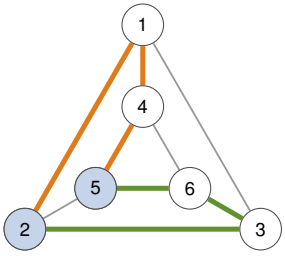
$B = 6, \quad T(X, Y) = 2$ för alla 15 par

Korrekthet: $\Rightarrow$ (ja blir ja)

Anta att G har en hamiltoncykel C.

- Välj delgrafen $G' = (V, E')$ där E' är precis kanterna i C.
- G' har $|V|$ kanter. Budgeten $B = |V|$ håller.
- Ta två hörn X och Y. De delar cykeln i två bågar, och bågarna är två disjunkta stigar mellan X och Y. Kravet $T(X, Y) = 2$ är uppfyllt.
- Alltså är (G, T, B) en ja-instans av INTERNETBESLUT.

Det här är den lätta riktningen. Det är den nästan alltid.



C: 1–2–3–6–5–4–1

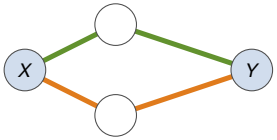
$X = 2$, $Y = 5$: 2–3–6–5 och 2–1–4–5

Korrekthet: $\Leftarrow$ (nej blir nej)

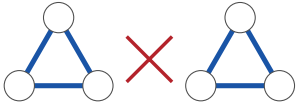
Anta att det finns en delgraf G' med $|V|$ kanter där alla par har två disjunkta stigar.

1. Varje hörn har grad minst 2 i G': två disjunkta stigar från X måste lämna X längs olika kanter.
2. Gradsumman är $2|E'| = 2|V|$. Alla grader är alltså *exakt* 2.
3. G' är sammanhängande, för det finns stigar mellan alla par.
4. En sammanhängande graf där alla hörn har grad 2 är en enda cykel genom alla hörn. G' är en hamiltoncykel i G. □

Anton Grensjös bilder (2017) frågar: unionen av två disjunkta stigar är en cykel, men varför går den genom *alla* hörn? Svaret är gradräkningen i steg 2.



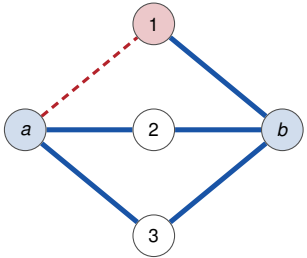
två stigar, två olika kanter ut ur X



sex hörn, sex kanter, alla grader 2 –
men inte sammanhängande. Faller på steg 3.

Exempel: det är budgeten som gör jobbet

- $K_{2,3}$: fem hörn, sex kanter.
- Med alla sex kanterna har varje par två disjunkta stigar. Nätet tål en trasig dator var som helst.
- Reduktionen sätter $B = |V| = 5$. En kant måste alltså bort – och då får ett hörn grad 1.
- Svaret blir nej, och det ska det bli: $K_{2,3}$ har ingen hamiltoncykel. (Den är bipartit med ett udda antal hörn.)
- Hade reduktionen satt $B = 6$ hade svaret blivit ja – och fel. Budgeten ska vara precis $|V|$.



utan kanten a–1 har hörn 1 bara en granne: en enda stig till alla andra

- **Talet B är hela reduktionen.** Med $B < |V|$ räcker kanterna inte till grad 2 överallt: alltid nej. Med $B > |V|$ godkänns grafer utan hamiltoncykel.
- Med $B = |V| + 1$ godkänns 20 av de 1 024 graferna med fem numrerade hörn fast de saknar hamiltoncykel. Det minsta motexemplet är $K_{2,3}$.
- **Det som inte spelar roll** är om ”bestående av B kanter” läses som exakt B (Anton Grensjö, Lisa Li) eller högst B (Alice Heavey).
- Fler kanter förstör aldrig en stig. En lösning med färre kanter kan fyllas på till exakt B så länge $B \leq |E|$: samma ja-instanser.
- Beviset för $\Leftarrow$ behöver bara ”högst $|V|$ kanter”. Att de är minst $|V|$ ger gradräkningen själv.

$B < |V|$
för snålt: alltid nej

$B = |V|$
ja precis när G har en hamiltoncykel

$B > |V|$
för generöst: ja även för $K_{2,3}$

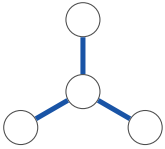
Utan budget, $B = |E|$, är problemet lätt: ta alla kanter och räkna maxflöden.

Kaninhål: går det med hamiltonstig i stället?

- HAMILTONSTIG: finns en stig som passerar varje hörn exakt en gång? Också känt NP-fullständigt, så försöket är rimligt.
- **Frestande:** en stig har $|V| - 1$ kanter och håller ihop alla hörn. Skicka G vidare oförändrad med $T \equiv 1$ och $B = |V| - 1$.
- **Fel.** Varje spännande träd godkänns. Stjärnan ger ja men saknar hamiltonstig. Fel svar för 4 av 64 grafer med fyra hörn och 95 av 1 024 med fem.
- Varningsklockan: med $T \equiv 1$ är problemet lätt. En korrekt reduktion dit vore ett bevis för $\mathrm{P} = \mathrm{NP}$.
- Andra tal hjälper inte. En stig klarar aldrig ett krav $T(X, Y) \geq 2$, och allt en stig klarar, klarar varje spännande träd med lika många kanter. Stigen måste bli en ring.



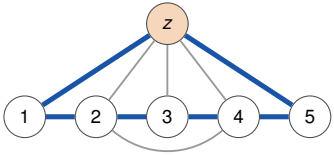
en hamiltonstig: 3 kanter, allt hänger ihop



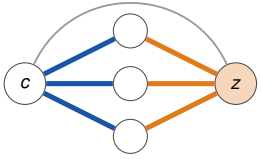
en stjärna: 3 kanter, allt hänger ihop.
Samma svar – men ingen hamiltonstig.

```
function Hamiltonstig(G = (V, E))
    H ← G + nytt hörn z, granne med alla i V
    B ← |V| + 1
    foreach par X, Y in H do
        T(X, Y) ← 2
    return InternetBeslut(H, T, B)
```

- $\Rightarrow$: en hamiltonstig $v_1, \ldots, v_n$ i G sluts till ringen $z, v_1, \ldots, v_n, z$ i H. Den har $|V| + 1$ kanter.
- $\Leftarrow$: beviset från förut ger en hamiltoncykel i H. Ta bort z ur den: kvar blir en hamiltonstig i G.
- Budgeten är antalet hörn i H, alltså $|V| + 1$. Med en kant till går det fel: stjärnan plus z innehåller $K_{2,3}$.
- Givna ändpunkter s och t? Ge z bara kanterna till s och t.



ringen z–1–2–3–4–5–z: sex kanter, $B = 6$



stjärnan (blå) plus z: sex kanter bildar $K_{2,3}$.
Med $B = 6$ blir svaret ja, utan hamiltonstig.

Kaninhål: räcker kanterna som certifikat?

- Steg 1 lade stigarna i certifikatet, som Anton Grensjö (2017). Det säkra valet: verifieraren kontrollerar, den letar aldrig.
- Lisa Li (2019) låter lösningen vara enbart G'. Då ska verifieraren själv avgöra om stigarna finns.
- **Invändning:** Ford–Fulkerson räknar ut ett flöde, inga stigar. Och stigarna algoritmen går längs under arbetet behöver inte vara disjunkta.
- Båda påståendena är sanna. Ändå räcker kanterna:

Sats

Givet G', X och Y kan man på polynomisk tid räkna ut det största antalet disjunkta stigar mellan X och Y – och stigarna själva.

Certifikat A
E' och alla stigarna
verifieraren kontrollerar stigarna

Certifikat B
bara E'
verifieraren får hitta stigarna själv

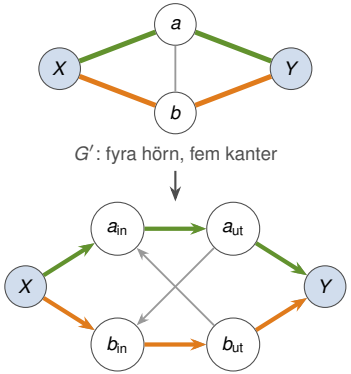
Båda duger. B behöver satsen.

Beviset tar tre bilder: stigar ger flöde, flöde ger stigar, och ett exempel där algoritmens egna stigar korsar varandra.

Bygg ett flödesnät N av G' för paret (X, Y):

1. X blir källa och Y utlopp. De delas inte: X får bara bågar ut, Y bara bågar in.
2. Varje annat hörn v blir två noder med en båge $v_{\text{in}} \to v_{\text{ut}}$.
3. Varje kant $\{u, v\}$ blir två bågar: $u_{\text{ut}} \to v_{\text{in}}$ och $v_{\text{ut}} \to u_{\text{in}}$.
4. Alla bågar får kapacitet 1.

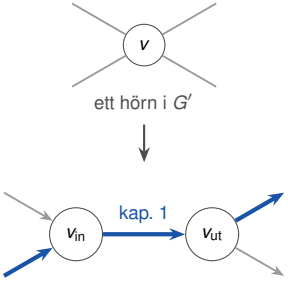
- **Lätta hållet:** k disjunkta stigar i G' ger ett flöde av storlek k i N. Skicka en enhet längs varje stig. Stigarna delar varken hörn eller kanter, så ingen kapacitet överskrids.
- Maxflödet är alltså *minst* antalet disjunkta stigar. Kan det vara större?



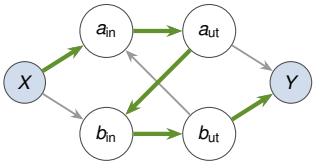
Ford–Fulkerson ger ett maxflöde f i N. Säg att det har värdet k.

1. **Heltal.** Flödet börjar på 0 och varje utökning ändrar det med ett heltal. Alltså: 0 eller 1 på varje båge.
2. **En enhet per nod.** Bågen $v_{\text{in}} \to v_{\text{ut}}$ rymmer 1. Den är enda vägen ut ur v_{in} och enda vägen in i v_{ut}. Varje nod utom X och Y har därför högst en flödesbåge in, och lika många ut.
3. **Följ flödet.** Börja i X och gå längs flödesbågar. Varje nod på vägen har precis en flödesbåge ut. Ingen nod nås två gånger, för den hade då haft två flödesbågar in. Nätet är ändligt, så vandringen tar slut – och det kan den bara göra i Y.
4. k **stigar.** Ur X går k flödesbågar. De ger k vandringar, som av samma skäl aldrig möts före Y. Slå ihop v_{in} och v_{ut} igen: k disjunkta stigar i G'. □

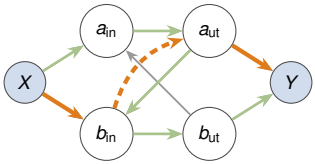
Med det lätta hållet från förra bilden: maxflödet är precis det största antalet disjunkta stigar, och stigarna står att läsa i flödet. Flödesbågar som blir över bildar slutna cykler och kastas.



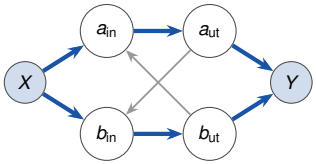
blått: bågar med flöde 1.
Högst en in, och då precis en ut.



första utökande stigen:
$X \to a \to b \to Y$



andra utökande stigen:
$X \to b$, bakåt till a, $a \to Y$



det färdiga flödet:
$X \to a \to Y$ och $X \to b \to Y$

- Ford–Fulkerson med djupetförstsökning på nätet N från exemplet. Första sökningen råkar gå $X \to a \to b \to Y$.
- Andra sökningen går $X \to b$, sedan *bakåt* över bågen $a_{\text{ut}} \to b_{\text{in}}$ och ut via $a \to Y$. Algoritmens två stigar går båda genom a och b. Invändningen stämmer.
- Men bakåtsteget tog bort flödet mellan a och b. Kvar står två disjunkta stigar.
- Stigarna läses av ur det **färdiga flödet**, inte ur algoritmens historia. Det är samma sak som i labb 3: de utökande stigarna backar över matchade kanter, men matchningen läses av ur flödet.

Kaninhål: vad betyder det för NP-beviset?

- **Verifierare med bara kanterna:** bygg N för varje par, räkna maxflödet och jämför med $T(X, Y)$. Stigarna behöver aldrig skrivas ut.
- **Tid:** högst $|V|^2$ par. Maxflödet är högst $|V| - 1$, så det blir högst $|V|$ sökningar à $O(|V| + |E|)$ per par. Polynomiskt.
- **Båda certifikaten duger.** Med stigarna i certifikatet behövs ingen flödesteori. Med bara kanterna vilar beviset på satsen.
- På en tenta: välj det certifikat vars verifiering du kan bevisa. Certifikatet får innehålla hur mycket hjälp som helst, bara det är polynomiskt långt.
- Med satsen om maxflöde och minsta snitt följer dessutom Mengers sats från början av avsnittet.

Certifikat A

E' och alla stigarna

verifieraren kontrollerar stigarna.

Ingen flödesteori behövs.

Certifikat B

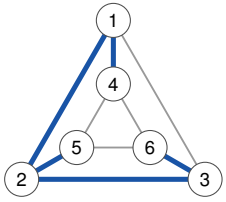
bara E'

verifieraren räknar ett maxflöde per par.

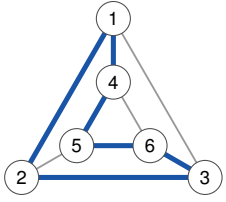
Vilar på satsen om flöde och stigar.

- **Hörndisjunkta eller kantdisjunkta stigar?** Uppgiften säger ”helt disjunkta (utom ändpunkterna)”, alltså hörndisjunkta. Reduktionen fungerar ordagrant också för kantdisjunkta stigar: gradargumentet är detsamma. Flödesbeviset likaså, utan hörndelning: stryk flöde som går åt båda hållen på samma kant, dra ut en vandring i taget och korta av slingorna.
- **Riktigt små grafer.** En graf med ett eller två hörn har ingen hamiltoncykel, och ett ensamt hörn är en hamiltonstig som inte kan slutas till en ring. En karpreduktion måste ge rätt svar för *alla* instanser, så ett pedantiskt bevis tar de minsta graferna för sig och returnerar en fix instans.
- **Jättestora $T(X, Y)$.** Ett par kan ha högst $|V| - 1$ disjunkta stigar, en per granne. Kräver indata $T(X, Y) = 10^{100}$ är svaret nej, och nej-instanser behöver inga certifikat. Certifikatet med stigar är alltså polynomiskt långt.
- **Kontrollräknat.** Ett program har prövat cykelreduktionen på samtliga 33 864 grafer med tre till sex numrerade hörn, stigreduktionen på samtliga 33 866 med två till sex, och flödessatsen mot uttömmande sökning på 8 660 par i slumpgrafer. Rätt varje gång. Det är inget bevis, men det är betryggande.

- $T \equiv 1$: varje par ska bara hänga ihop. Ett spännande träd räcker, så svaret är ja precis när G är sammanhängande och $|V| - 1 \leq B \leq |E|$. Linjär tid med BFS.
- $T \equiv 2$: NP-fullständigt. Det har vi just visat.
- Från träd till cykel, från lätt till svårt. Samma språng kommer tillbaka i uppgift 4: två tidsenheter är lätt, tre är svårt.
- I litteraturen heter problemet *survivable network design*. Varianten med hörndisjunkta stigar kallas VC-SNDP, den med kantdisjunkta EC-SNDP.
- För EC-SNDP finns en algoritm som alltid hamnar inom en faktor 2 från det billigaste nätet (Jain, 2001). Sådana approximationsalgoritmer är ämnet för nästa övning.



$T \equiv 1$: ett spännande träd, 5 kanter

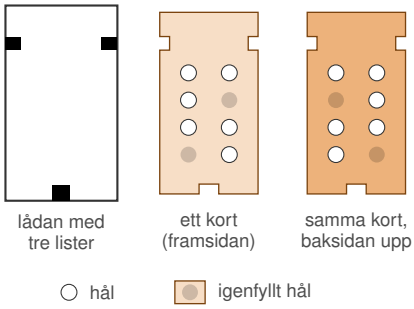


$T \equiv 2$: en hamiltoncykel, 6 kanter

Uppgift 3: Håstads leksaksaffär

Uppgift 3: Håstads leksaksaffär

- Håstads leksaksaffär säljer ett pussel som består av en låda och ett antal kort.
- Ett kort kan placeras i lådan på två sätt, med framsidan upp eller med baksidan upp, eftersom det finns urgröpningar i korten som måste passa in i lister som sitter i lådan.
- På varje kort finns två kolumner med hål, men vissa av hålen är igenfyllda.
- **Mål:** lägg korten i lådan så att hela botten täcks – för varje hålposition måste alltså åtminstone ett av korten ha ett igenfyllt hål.



Uppgift

Bevisa att språket $\textsc{Toys} = \{\langle c_1, c_2, \ldots, c_n\rangle : (c_i \text{ beskriver kort}) \wedge (\text{det går att lösa pusslet})\}$ är NP-fullständigt.

Pusslet finns på riktigt



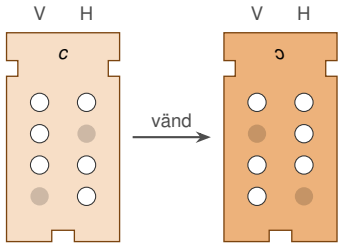


Foto ur Anton Grensjös övningsbilder (2017). Till höger: lådan med sina lister och ett kort på plats.

- Fyra kort med tre rader och två kolumner. Det fjärde har hål i bara den ena kolumnen. Lägg det på minnet.
- Listerna i lådan passar i urgröpningarna. Ett kort går därför bara att lägga på två sätt.
- **Att vända ett kort är att spegla det:** vänster och höger kolumn byter plats. Raderna ligger kvar.
- Det är allt pusslet handlar om: ett binärt val per kort.

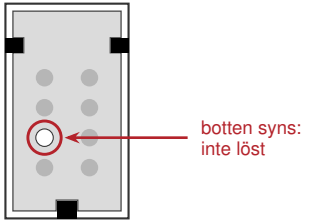
Formalisering: vad är ett kort, och vad är en lösning?

- Ett kort med m rader beskrivs av två bitvektorer med m bitar var: **vänsterkolumnen** och **högerkolumnen**. 1 betyder igenfyllt, 0 betyder hål.
- Ett vänt kort är samma kort med kolumnerna ombytta.
- En **lösning** är ett val per kort: rättvänt eller vänt. Med n kort finns 2^n sätt att lägga dem.
- En position (rad j, kolumn vänster eller höger) är **täckt** om minst ett av korten, så som det ligger, är igenfyllt där.
- Pusslet är löst när alla $2m$ positioner är täckta. Ordningen i högen spelar ingen roll.



Steg 1: Toys ligger i NP

- **Lösning (certifikat):** en beskrivning av hur korten ska läggas, alltså en bit per kort.
- **Verifiera:** lägg korten i lådan enligt beskrivningen och titta efter om man ser botten.
- Facit: det räcker med en blick. För en dator är blicken en dubbelslinga: för var och en av de $2m$ positionerna, finns det ett kort som är igenfyllt där?
- **Tid:** $O(n \cdot m)$, linjärt i indatas storlek. Indata är n kort med $2m$ bitar var.
- Certifikatet är n bitar långt. Här finns inga tal att bli lurad av, så pseudopolynomiska fällor saknas helt.



Steg 2: vilket känt problem liknar det här?

- Ett kort kan ligga på två sätt.
  En boolesk variabel kan anta två värden.
- Varje position ska täckas av *minst ett* kort.
  Varje klausul ska göras sann av *minst en* literal.
- ”Ett lämpligt, om än skrämmande val, är CNF-SAT.”
  (Alice Heavey, 2019)

CNF-SAT

Indata: en formel på konjunktiv normalform,

$$\varphi = (l_1 \vee l_2 \vee \cdots \vee l_i) \wedge (l_j \vee \cdots \vee l_k) \wedge \ldots$$

med n variabler och m klausuler. Varje literal l är en variabel x_i eller dess negation $\overline{x}_i$.

Fråga: finns det en tilldelning av variablerna som gör φ sann?

Mål: givet φ, bygg en korthög γ så att φ är satisfierbar $\iff$ pusslet γ går att lösa.

Reduktionen: från formel till kort

Löpande exempel: $\varphi = (x_1 \vee \overline{x}_2) \wedge (\overline{x}_1 \vee \overline{x}_2 \vee \overline{x}_4) \wedge (x_2 \vee \overline{x}_4) \wedge (\overline{x}_3 \vee x_4)$, med $n = 4$ variabler och $m = 4$ klausuler.

a) Variabeln x_i motsvaras av kortet c_i.

b) Varje kort har två kolumner med m positioner: en rad per klausul.

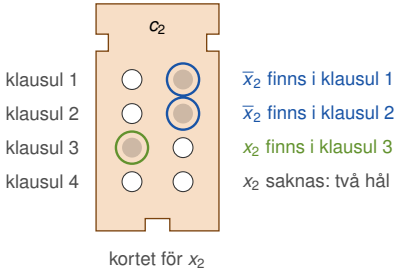
c) Om x_i förekommer i klausul j är **vänstra** hålet på rad j igenfyllt.

d) Om $\overline{x}_i$ förekommer i klausul j är **högra** hålet på rad j igenfyllt.

e) Alla andra positioner är hål.

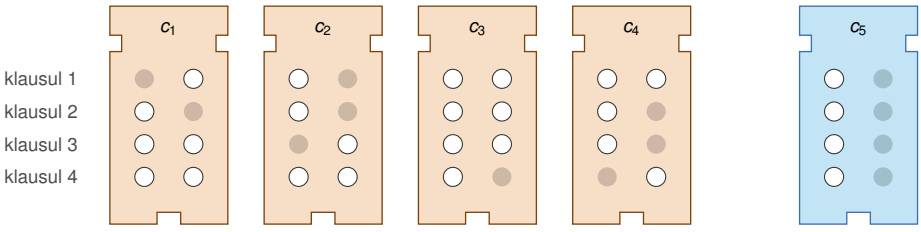
f) Dessutom ett extra, ”magiskt” kort c_{n+1} med hål bara i vänsterkolumnen. Högerkolumnen är helt igenfylld.

Tanken: c_i rättvänt betyder $x_i = 1$, vänt betyder $x_i = 0$.



Exempel: en formel blir fem kort

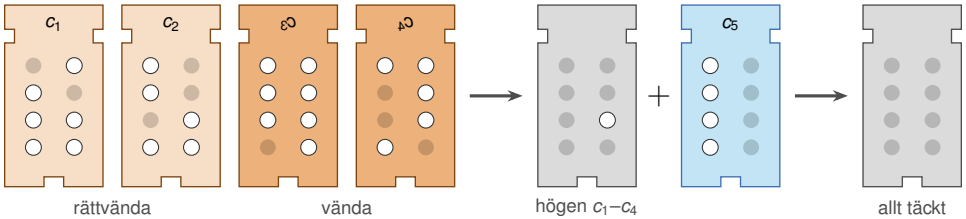
$$\varphi = (x_1 \vee \overline{x}_2) \wedge (\overline{x}_1 \vee \overline{x}_2 \vee \overline{x}_4) \wedge (x_2 \vee \overline{x}_4) \wedge (\overline{x}_3 \vee x_4)$$



Rad j är klausul j. Igenfyllt till vänster: x_i finns i klausulen. Igenfyllt till höger: $\overline{x}_i$ finns i klausulen.

Exempel: en tilldelning blir en läggning

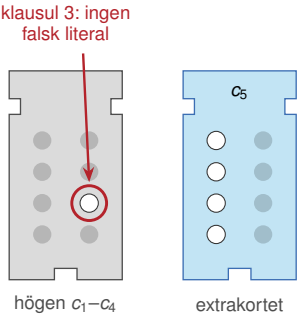
$\varphi = (x_1 \vee \overline{x}_2) \wedge (\overline{x}_1 \vee \overline{x}_2 \vee \overline{x}_4) \wedge (x_2 \vee \overline{x}_4) \wedge (\overline{x}_3 \vee x_4)$ satisfieras av $x_1 = 1$, $x_2 = 1$, $x_3 = 0$, $x_4 = 0$.



- $x_1 = x_2 = 1$: korten c_1 och c_2 läggs rättvända.
- $x_3 = x_4 = 0$: korten c_3 och c_4 vänds. Deras högerkolumner hamnar till vänster.
- I högen är hela vänsterkolumnen täckt. Till höger finns ett hål kvar, på rad 3.
- Extrakortet täpper till högerkolumnen. Pusslet är löst.

Varför behövs extrakortet?

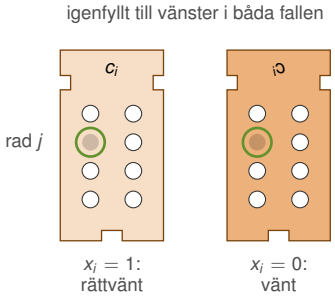
- **Vänster** på rad j täcks av variabelkorten precis när klausul j innehåller en **sann** literal. Det är vad reglerna är byggda för.
- **Höger** på rad j täcks av variabelkorten precis när klausul j innehåller en **falsk** literal. Det är spegelbilden, och den fick vi på köpet.
- Men en satisfierande tilldelning får gärna göra *alla* literaler i en klausul sanna. I exemplet: klausul 3 är $(x_2 \vee \overline{x}_4)$, med $x_2 = 1$ och $x_4 = 0$.
- Extrakortet tar bort det oönskade kravet. Rättvänt täcker det hela högerkolumnen och ingenting till vänster.
- Alice Heavey (2019): hålet till höger visar vad den negerade tilldelningen hade gett, och den behöver inte lösa formeln.



Korrekthet: $\Rightarrow$ (satisfierbar formel ger lösbart pussel)

Anta att φ satisfieras av en tilldelning av $x_1, \ldots, x_n$.

- Lägg c_i rättvänt om $x_i = 1$ och vänt om $x_i = 0$. Lägg extrakortet rättvänt.
- **Vänsterkolumnen, rad j.** Klausul j innehåller en sann literal.
  - Är den x_i (alltså $x_i = 1$) ligger c_i rättvänt, och dess vänstra hål på rad j är igenfyllt.
  - Är den $\overline{x}_i$ (alltså $x_i = 0$) är c_i vänt, och dess igenfyllda högerhål på rad j ligger nu till vänster.
- **Högerkolumnen.** Extrakortet täcker den helt.
- Alla positioner är täckta. Pusslet γ är löst. $\square$



Korrekthet: $\Leftarrow$, fall 1 – extrakortet ligger rättvänt

Anta att pusslet γ är löst, och att extrakortet ligger rättvänt.

- Extrakortet täcker ingenting i vänsterkolumnen. Variabelkorten måste alltså täcka den själva.
- Läs av en tilldelning ur läggningen:

$$x_i = \begin{cases} 1 & \text{om } c_i \text{ ligger rättvänt} \\ 0 & \text{annars.} \end{cases}$$

- **Rad j, vänster.** Något kort c_i är igenfyllt där, så som det ligger.
  - c_i rättvänt med vänstra hålet igenfyllt: x_i finns i klausul j, och $x_i = 1$.
  - c_i vänt med (sitt eget) högra hål igenfyllt: $\overline{x}_i$ finns i klausul j, och $x_i = 0$.

  I båda fallen är klausul j sann.
- Alla m klausuler är sanna. φ är satisfierad.

Korrekthet: $\Leftarrow$, fall 2 – extrakortet ligger vänt



löst, men extrakortet är vänt
avläst: $x = (0, 0, 1, 1)$, som *inte* satisfierar φ

fortfarande löst, och nu är det fall 1
avläst: $x = (1, 1, 0, 0)$, som satisfierar φ

- Nu täcker extrakortet hela vänsterkolumnen och ingenting till höger. Variabelkorten har täckt *högerkolumnen*, det vill säga löst den speglade uppgiften.
- **Vänd hela högen.** Ett löst pussel förblir löst när alla kort vänds: hela bilden speglas, och en täckt botten är täckt också i spegeln.
- Efter vändningen ligger extrakortet rätt, och fall 1 ger en satisfierande tilldelning.
- Uttryckt i variabler: läs av tilldelningen som förut och **invertera** den. Det är facits formulering – och svaret på Antons fråga ”vad gör vi om extrakortet är vänt?”. □

Polynomisk tid – och klart

- Reduktionen bygger $n + 1$ kort med $2m$ positioner var. För varje literal i φ fylls ett hål igen. Tid $O(n \cdot m)$.
- Både n och m är högst längden av φ, så tiden är polynomisk i indatas storlek.
- Vi har visat:
  1. TOYS $\in$ NP,
  2. CNF-SAT $\leq_{\mathrm{p}}$ TOYS, med en reduktion som är polynomisk och korrekt åt båda håll.
- TOYS är NP-fullständigt.

Vad var det svåra?

Inte beviset. Det svåra var att se att ett kort är en variabel, att en rad är en klausul – och att spegelbilden behövde ett extra kort för att inte ställa till det.

Kaninhål: vad är pusslet egentligen?

- Stryk extrakortet. Då måste variabelkorten täcka *båda* kolumnerna själva.
- Vänster på rad j är täckt $\iff$ klausul j har en sann literal.
  Höger på rad j är täckt $\iff$ klausul j har en falsk literal.
- Pusslet utan extrakort är alltså lösbart precis när det finns en tilldelning där varje klausul innehåller *både* en sann och en falsk literal. Det problemet har ett namn: NOT-ALL-EQUAL-SAT, NAE-SAT. Det är också NP-fullständigt.
- Det gäller åt båda håll. *Varje* korthög är en NAE-SAT-instans: läs av korten rad för rad, igenfyllt till vänster som x_i och igenfyllt till höger som $\overline{x}_i$.
- Extrakortet är standardknepet för att reducera SAT till NAE-SAT: lägg en ny variabel z i varje klausul, här i formen $\overline{z}$. Symmetrin ”vänd alla kort” är NAE-SATs egen symmetri ”invertera alla variabler”.
- Håstads leksaksaffär säljer alltså NAE-SAT i trälåda.



- Det fjärde kortet har hål i bara en kolumn. Det är extrakortet, och kolumnen med hålen är den vi kallar vänster.
- Läs av rad för rad. Igenfyllt till vänster: variabeln. Igenfyllt till höger: negationen.
  - rad 1: a och b
  - rad 2: $\overline{a}$, $\overline{b}$ och c
  - rad 3: b och $\overline{c}$
- $\varphi = (a \vee b) \wedge (\overline{a} \vee \overline{b} \vee c) \wedge (b \vee \overline{c})$
- Fyra tilldelningar satisfierar φ. Den enklaste är $a = b = c = 1$: lägg korten precis som de ligger på fotot. Vill man vända på något: $a = 0$, $b = 1$, $c = 0$ går också.

- **Spelar ordningen i högen någon roll?** Nej. En position är täckt om *något* kort är igenfyllt där.
- **Kan x_i och $\overline{x}_i$ stå i samma klausul?** Ja. Då är båda hålen igenfyllda på den raden av c_i, och raden är täckt hur kortet än ligger. Klausulen är också sann hur x_i än väljs. Det stämmer.
- **En tom klausul?** Då har inget variabelkort något igenfyllt hål på den raden, och extrakortet täcker bara den ena sidan. Pusslet går inte att lösa, och formeln är inte satisfierbar. Det stämmer också.
- **Varför CNF-SAT och inte 3-CNF-SAT?** Båda duger. Reduktionen bryr sig inte om hur många literaler en klausul har, och 3-CNF-SAT är ett specialfall.
- **Var kommer pusslet ifrån?** Det är en klassiker. Samma uppgift finns under namnet PUZZLE i Sipsers lärobok *Introduction to the Theory of Computation*.

Uppgift 4: Processorschemaläggning

Uppgift 4: Processorschemaläggning

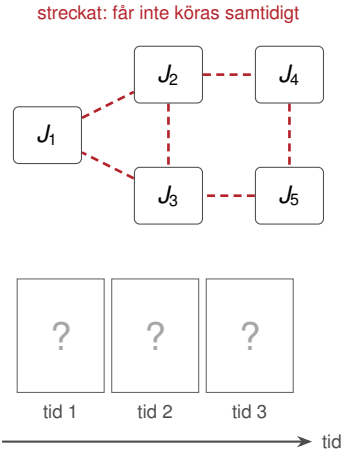
- Givet: n jobb som ska exekveras och ett tillräckligt antal processorer.
- Varje jobb tar en tidsenhet att utföra.
- Det finns också villkor på formen *jobb i kan inte exekveras samtidigt som jobb j*.
- SCHEDULE: går det att schemalägga jobben så att alla jobb kan exekveras på tre tidsenheter?

Uppgift

a) Visa att SCHEDULE är NP-fullständigt.

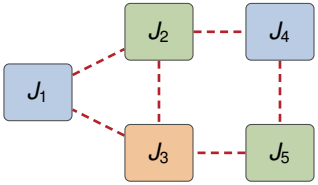
b) Är problemet fortfarande NP-fullständigt om vi kräver att jobben ska vara exekverade efter k tidsenheter där $k > 3$?

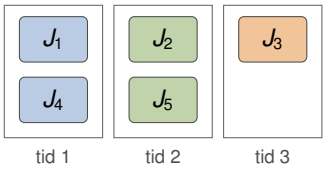
c) Är det sant för $k = 2$?



Rita problemet: konfliktgrafen

- Låt varje jobb vara ett hörn.
- Låt varje villkor ”jobb *i* inte samtidigt som jobb *j*” vara en kant mellan *i* och *j*.
- Ett schema ger varje jobb en tidsenhet. Två jobb med ett villkor mellan sig måste få olika.
- Byt ordet ”tidsenhet” mot ”färg” och läs meningen igen.
- Ett schema på tre tidsenheter *är* en trefärgning av konfliktgrafen. Processorerna spelar ingen roll: det finns alltid tillräckligt många.





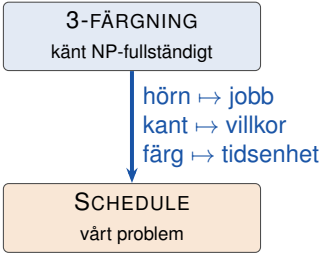
Steg 1: SCHEDULE ligger i NP

- Lisa Lis bilder (2019) lämnar det här steget som egen övning. Här är den.
- **Lösning (certifikat):** ett schema, det vill säga en tidsenhet $t(i) \in \{1, 2, 3\}$ för varje jobb i.
- **Verifiera:**
  a) varje jobb har fått precis en tidsenhet mellan 1 och 3,
  b) för varje villkor (i, j) gäller $t(i) \neq t(j)$.
- **Tid:** $O(n +$ antalet villkor$)$. Linjärt i indatas storlek.
- Vi behöver aldrig kontrollera att processorerna räcker. Uppgiften lovar att de gör det, och n stycken räcker alltid.

Steg 2: reducera 3-FÄRGNING

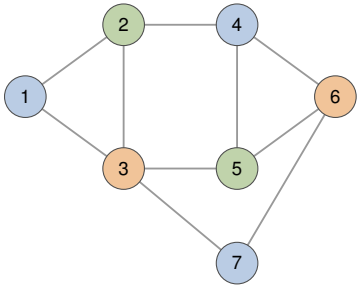
```
function 3-färgning(G = (V, E))
    foreach v in V do
        skapa ett jobb v
    foreach (u, v) in E do
        skapa villkoret "u inte samtidigt som v"
    return Schedule(jobben, villkoren)
```

- 3-FÄRGNING är känt NP-fullständigt, och det är dessutom samma problem som vårt. Det är ett bra val.
- Ett anrop, svaret returneras direkt: en karpreduktion.
- Reduktionen går igenom varje hörn och varje kant en gång: $O(|V| + |E|)$.



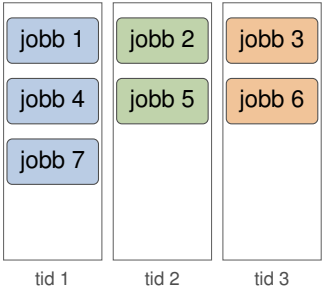
Exempel: sju hörn blir sju jobb

grafen G: 7 hörn, 10 kanter



reduktion

schemat: 7 jobb, 10 villkor, 3 tidsenheter



- En trefärgning: hörnen 1, 4 och 7 får färg 1; hörnen 2 och 5 färg 2; hörnen 3 och 6 färg 3.
- Samma sak som schema: jobben 1, 4 och 7 körs under tidsenhet 1, och så vidare. Inga två jobb i samma kolumn har ett villkor mellan sig.
- Två tidsenheter räcker inte: hörnen 1, 2 och 3 bildar en triangel.

Korrekthet: en trefärgning är ett schema, och omvänt

$\Rightarrow$ G trefärgbar ger schema på 3 tidsenheter

- Ta en trefärgning och låt C_i vara hörnen med färg i.
- Det finns inga kanter inom C_i, alltså inga villkor mellan jobben i C_i.
- Kör alla jobb i C_1 under tidsenhet 1, alla i C_2 under tidsenhet 2 och alla i C_3 under tidsenhet 3.

$\Leftarrow$ schema på 3 tidsenheter ger trefärgning

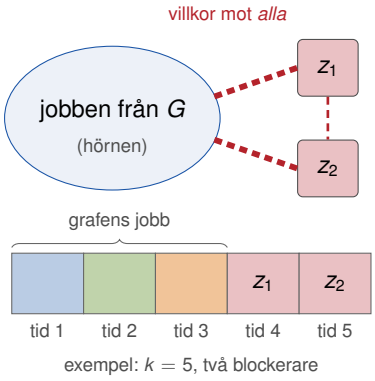
- Ge hörnet v färg i om jobbet v körs under tidsenhet i.
- Två grannar har ett villkor mellan sig. Deras jobb körs alltså under olika tidsenheter, och hörnen får olika färg.
- Det är en giltig trefärgning.

SCHEDULE ligger i NP och 3-FÄRGNING $\leq_p$ SCHEDULE. Alltså är SCHEDULE NP-fullständigt. □

Beviset är kort eftersom reduktionen inte gör något. De två problemen är samma problem i olika kläder.

b) Fler tidsenheter: $k > 3$

- **Facit:** med samma argument kan k-FÄRGNING reduceras till SCHEDULE med k tidsenheter. Alltså NP-fullständigt för varje $k \geq 3$.
- Det förutsätter att k-FÄRGNING är NP-fullständigt för varje fast $k \geq 3$. Är det det?
- **Direkt variant:** reducera 3-FÄRGNING igen, men lägg till $k - 3$ *blockerarjobb* $z_1, \ldots, z_{k-3}$ med villkor mot alla andra jobb, och mot varandra.
- Varje blockerare tvingas ensam in i en egen tidsenhet. Kvar till grafens jobb blir precis tre.
- G trefärgbar $\iff$ jobben går på k tidsenheter. Samma knep, med en klick på $k - 3$ hörn, ger att k-FÄRGNING är NP-fullständigt.

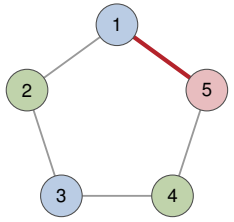


c) Två tidsenheter: $k = 2$

Nu vänder vi på reduktionen och *löser* problemet: bygg konfliktgrafen och försök tvåfärga den.

```
function Schedule2(jobb, villkor)
    foreach jobb s utan tid do
        tid[s] ← 1;  lägg s i kön
        while kön inte är tom do
            u ← ta ut ett jobb ur kön
            foreach villkor (u, v) do
                if v saknar tid then
                    tid[v] ← 3 - tid[u];  lägg v i kön
                else if tid[v] = tid[u] then
                    return nej
    return ja
```

- Har ett jobb tid 1 måste alla dess grannar ha tid 2, och tvärtom. Inget val finns att göra.
- Bredden-först-sökning (facit tar djupet-först, det går lika bra): $O(n +$ antalet villkor$)$. Problemet ligger i P.



tider 1, 2, 1, 2 – och jobb 5 krockar med både jobb 4 och jobb 1

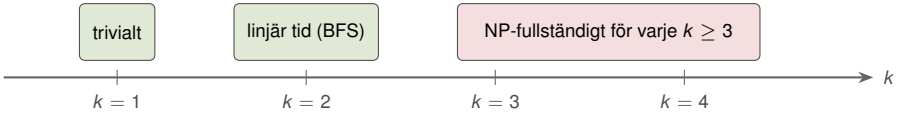
udda cykel: går inte på två tidsenheter. Utan udda cykler går det alltid (grafen är bipartit).

Samma översättning, motsatt håll, motsatt slutsats

	Reduktion	**Vad den visar**
$k = 3$	3-FÄRGNING $\leq_{\mathrm{p}}$ SCHEDULE	SCHEDULE är minst lika svårt som 3-FÄRGNING: NP-svårt
$k = 2$	$\text{SCHEDULE}_2 \leq_{\mathrm{p}}$ 2-FÄRGNING	SCHEDULE_2 är inte svårare än 2-FÄRGNING: ligger i P

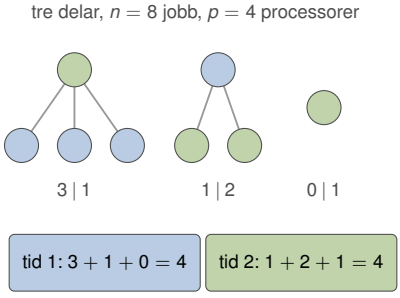
- Översättningen är densamma i båda fallen: jobb är hörn, villkor är kanter.
- För att visa att ett problem är **svårt** reducerar man ett svårt problem *till* det.
- För att visa att ett problem är **lätt** reducerar man det till ett lätt problem. Det kallas vanligen att skriva en algoritm.
- Den som i tentasalen skriver ”vi reducerar SCHEDULE till 3-FÄRGNING, alltså är SCHEDULE NP-svårt” har visat att SCHEDULE ligger i NP. Det är också sant, men det var inte det som skulle visas.

- Uppgiftens svar: **nej**. Med två tidsenheter löses problemet på linjär tid.
- Pedantens svar: nej, *såvida inte* $\mathrm{P} = \mathrm{NP}$.
- Om $\mathrm{P} = \mathrm{NP}$ är nämligen varje problem i P NP-fullständigt, utom de två som alltid svarar ja respektive alltid svarar nej. Reduktionen från ett godtyckligt problem i NP: lös instansen på polynomisk tid, och skriv sedan ut en fix ja-instans eller en fix nej-instans.
- Därför säger en försiktig teoretiker ”ligger i P” och inte ”är inte NP-fullständigt”.
- Och $k = 1$? Alla jobb körs samtidigt. Svaret är ja precis när det inte finns några villkor alls.



Kaninhål: tänk om processorerna tar slut?

- Uppgiften lovar ”ett tillräckligt antal processorer”. Anta i stället att det finns p stycken: högst p jobb per tidsenhet.
- $k = 3$: fortfarande NP-fullständigt, förstås.
- $k = 2$: tvåfärga varje sammanhängande del av konfliktgrafen för sig. Del i har två sidor, med a_i och b_i jobb, och får vändas fritt.
- Välj en sida per del så att högst p jobb hamnar i varje tidsenhet. Det är en delmängdssumma!
- Men talen a_i och b_i är högst n. De *räknar* saker i indata i stället för att stå skrivna där. Dynamisk programmering över ”antal jobb i tidsenhet 1” tar $O(n^2)$, polynomiskt på riktigt.



Pseudopolynomiskt är bara ett problem när talen står i indata.

Kaninhål: samma problem i andra kläder

- **Tentaschema:** kurser är jobb, en gemensam student är ett villkor, tentatillfällen är tidsenheter.
- **Registerallokering i kompilatorer:** variabler är hörn, två variabler som lever samtidigt har en kant, processorns register är färger. (Chaitin m.fl., tidigt 1980-tal.)
- **Frekvenstilldelning:** sändare är hörn, sändare som stör varandra har en kant, frekvenser är färger.
- **Sudoku:** 81 hörn, nio färger, och några hörn färdigfärgade av tidningen.
- Det är samma NP-fullständiga problem varje gång. Namnet byts. Svårigheten består.

Praktisk nytta

Känner du igen ditt problem som graffärgning vet du två saker: att det finns färdiga heuristiker, och att du inte ska lova chefen en snabb exakt algoritm.

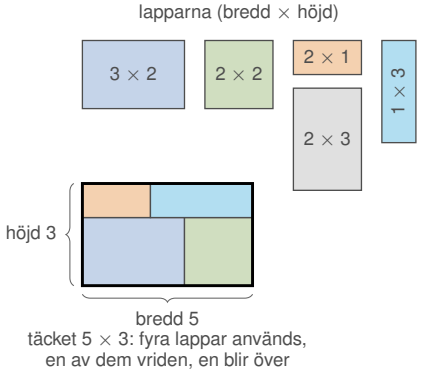
Uppgift 5: Lapptäcke

Uppgift 5: Lapptäcke

- *Patchwork*, från programvaruföretaget Stitcher, pusslar ihop rektangulära lappar till ett lapptäcke utan hål och utan lappar som överlappar varandra.
- **Indata:** en lista av rektangulära lappar (bredd och höjd i hela millimeter) och önskad storlek (bredd och höjd) på det färdiga rektangulära lapptäcket.
- **Utdata:** om en *delmängd* av lapparna kan pusslas ihop till ett täcke av rätt storlek – en beskrivning av hur. För varje lapp som används: var den ska placeras och om den behöver vridas.

Uppgift

Formulera detta konstruktionsoptimeringsproblem med matematisk notation och visa att det tillhörande beslutsproblemet (existerar en lösning?) är NP-fullständigt.



Formulering med matematisk notation

Indata till konstruktionsproblemet LK:

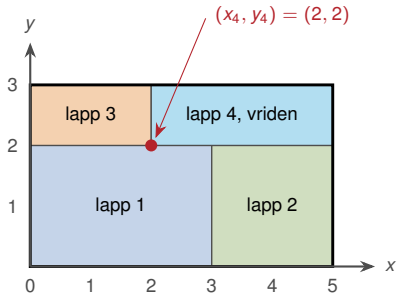
- en lista med talpar (b_i, h_i), $1 \leq i \leq n$: bredd och höjd på lapp i,
- två tal B och H: täckets bredd och höjd.

Samtliga tal är positiva heltal.

Lösning: en lista med fyrtupler (l_i, x_i, y_i, v_i), där

- $l_i \in \{1, \ldots, n\}$ är numret på en lapp,
- (x_i, y_i) är var lappens nedre vänstra hörn ska ligga,
- v_i är en boolesk variabel som anger om lappen ska vridas.

Villkor: lapparna får inte överlappa och måste täcka samtliga punkter (x, y) med $0 \leq x < B$ och $0 \leq y < H$. Inga andra punkter får täckas.



indata: $(3, 2),\ (2, 2),\ (2, 1),\ (1, 3),\ (2, 3)$
$B = 5,\ H = 3$

lösning: $(1, 0, 0, \text{nej}),\ (2, 3, 0, \text{nej}),$
$(3, 0, 2, \text{nej}),\ (4, 2, 2, \text{ja})$

Beslutsproblemet och planen

- LK: *konstruera* ett lapptäcke (eller säg att inget finns).
- LB: *finns* det en lösning till LK? Ja eller nej.
- För att visa att LB är NP-fullständigt visar vi
  1. att LB ligger i NP,
  2. att DELMÄNGDSSUMMA, ett känt NP-fullständigt problem, kan reduceras till LB.
- Steg 1 ser oskyldigt ut. Det är där fällan ligger.

En randanmärkning

Uppgiften säger ”konstruktionsoptimeringsproblem”, men här finns ingen målfunktion. Varje täcke av rätt storlek duger; inget är bättre än något annat.

Det gör ingenting. Vi ska ändå bara bevisa något om beslutsproblemet.

Steg 1, första försöket: rita upp täcket

- Naturlig idé: gör en $B \times H$-matris med en ruta per kvadratmillimeter. Gå igenom lapparna en i taget och markera rutorna de täcker.
- En ruta som redan är tagen: överlapp. En lapp utanför matrisen: utanför täcket. En omarkerad ruta på slutet: hål.
- Lätt att programmera, uppenbart korrekt.
- Och **pseudopolynomiskt**. Matrisen har $B \cdot H$ rutor, och det är exponentiellt många i indatas längd.
- Det hjälper inte om matrisen är själva certifikatet (Lisa Li, 2019) eller något verifieraren ritar själv (Alice Heavey, 2019). För stor är den ändå.

en omarkerad ruta: hål

3	3	4	4	?
1	1	1	2	2
1	1	1	2	2

matrisen för ett 5×3-täcke:
15 rutor, ett lappnummer i varje

Hur stort är täcket egentligen?

bredd = höjd	bitar	rutor i matrisen
1	2	1
10	8	100
100	14	10 000
1 000	20	1 000 000
10 000	28	100 000 000
100 000	34	10 000 000 000
10^9	60	10^{18}

Mått i millimeter. Bitar: bredd och höjd tillsammans.

De sex första raderna är Alice Heaveys tabell (2019).

- Talen B och H skrivs med $\log B + \log H$ bitar.
- Matrisen har $B \cdot H = 2^{\log B + \log H}$ rutor.
- Kvadratiskt i *talen* i indata. Exponentiellt i indatas *längd*.
- En siffra till i bredden och en till i höjden gör täcket hundra gånger större.
- Sista raden är ett lapptäcke på 1000 km gånger 1000 km. Det beskrivs med 60 bitar och har en triljon rutor. Ingen hinner pricka av dem.

Steg 1, andra försöket: räkna i stället för att rita

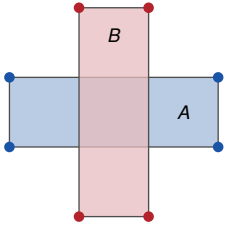
Lösning (certifikat): listan (l_i, x_i, y_i, v_i) från formuleringen. Högst n lappar, och varje tal ryms i lika många bitar som B eller H. Certifikatet är alltså polynomiskt långt.

Verifiera:

1. **Ingen lapp används två gånger:** pricka av lappnumren på en lista. $O(n)$.
2. **Varje lapp ligger inom täcket:** fyra jämförelser per lapp. $O(1)$ per lapp.
3. **Inga två lappar överlappar:** det finns $O(n^2)$ par, och ett par kontrolleras på $O(1)$ tid.
4. **Den sammanlagda ytan är $B \cdot H$:** en multiplikation per lapp och en summa. $O(n)$.

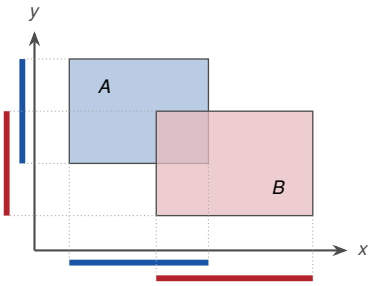
- Sammanlagt $O(n^2)$ räkneoperationer på tal med polynomiskt många bitar. Polynomisk tid. LB $\in$ NP.
- Vi rör aldrig en enda kvadratmillimeter. Två saker återstår att förklara: hur punkt 3 går på konstant tid, och varför punkt 4 räcker för att utesluta hål.

Punkt 3: överlappar två lappar?



Räcker det att kolla om något hörn av den ena lappen ligger i den andra? **Nej.** ”Tänk om lapparna ligger i ett kors!” (Lisa Li, 2019)

Här ligger inget av de åtta hörnen inuti den andra lappen.



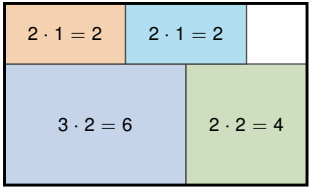
Med nedre vänstra hörnet (x_1, y_1) och övre högra (x_2, y_2) överlappar A och B precis när

$$A.x_1 < B.x_2 \wedge B.x_1 < A.x_2 \wedge A.y_1 < B.y_2 \wedge B.y_1 < A.y_2.$$

Fyra jämförelser, konstant tid (Alice Heavey, 2019). För korset är alla fyra sanna.

Punkt 4: varför räcker det att räkna ytan?

- Vi har redan kontrollerat att alla lappar ligger inom täcket och att inga två överlappar.
- Då täcker lapparna tillsammans precis så många kvadratmillimetrar som summan av deras ytor.
- Täcket har $B \cdot H$ kvadratmillimetrar. Är summan $B \cdot H$ finns ingen ruta över. **Inga hål.**
- Är summan mindre finns det ett hål någonstans. Var det ligger behöver vi inte veta.
- Lådprincipen, läst baklänges: lika många föremål som lådor, högst ett föremål per låda, alltså ingen tom låda.

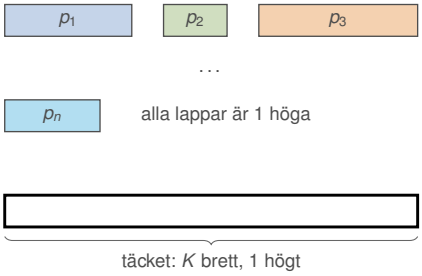


$6 + 4 + 2 + 2 = 14 < 15$ **hål**

Steg 2: reducera DELMÄNGDSSUMMA

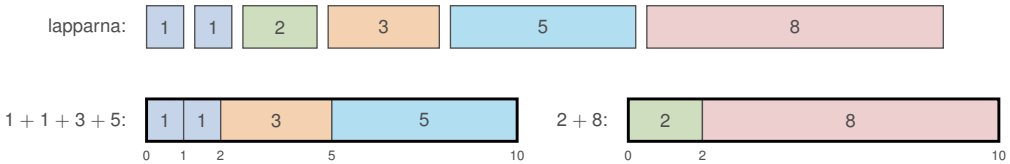
DELMÄNGDSSUMMA: givet positiva heltal $p_1, \ldots, p_n$ och K, finns det en delmängd av talen vars summa är K?

```
function Delmängdssumma(p_1, ..., p_n, K)
    L ← tom lista
    for i ← 1 to n do
        lägg till lappen (p_i, 1) i L
    return Lapptäcke(L, K, 1)
```



- Talet p_i blir en lapp som är p_i bred och 1 hög. Täcket ska bli K brett och 1 högt.
- Ett anrop, svaret returneras direkt: en karpreduktion.
- Talen kopieras som de står, i binär form. Reduktionen ritar ingenting och tar linjär tid i indatas längd.

Exempel: $\{1, 1, 2, 3, 5, 8\}$ och $K = 10$



- Alice Heaveys exempel (2019): delmängden $\{1, 1, 3, 5\}$ har summan 10. Lägg lapparna på rad, så blir det ett 10×1-täcke. De små talen under täcket är x-koordinaterna: delsummorna.
- Det finns fler: $2 + 8$, $2 + 3 + 5$ och $1 + 1 + 8$. Fyra delmängder, fyra täcken.
- Hade K varit 21 hade det inte funnits något täcke: alla sex lapparna tillsammans är bara 20 breda.

Korrekthet

$\Rightarrow$ delmängd med summan K ger täcke

- Ta lapparna som hör till delmängden.
- Lägg dem på rad från vänster till höger. Deras bredder har summan K.
- Resultatet är ett $K \times 1$-täcke utan hål och utan överlapp.

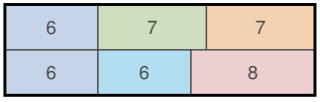
$\Leftarrow$ täcke ger delmängd med summan K

- Ta ett $K \times 1$-täcke byggt av några av lapparna.
- Lapp i har ytan p_i, vriden eller inte. Täcket har ytan K.
- Inga hål och inga överlapp: de använda lapparnas ytor har summan K. Motsvarande tal har summan K.

- Vridningen då? En vriden lapp är 1 bred och p_i hög. Den får plats i ett täcke som är 1 högt bara om $p_i = 1$, och då är den en kvadrat. Ytargumentet behöver inte ens bry sig.
- LB ligger i NP och DELMÄNGDSSUMMA $\leq_{\mathrm{p}}$ LB. Alltså är LB NP-fullständigt. □

- Kappsäck kan lösas pseudopolynomiskt. Kan lapptäcke det?
- **Starkt** NP-fullständigt: NP-fullständigt också när alla tal i indata är små (polynomiska i indatas längd). Då finns ingen pseudopolynomisk algoritm, om inte $\mathrm{P} = \mathrm{NP}$.
- 3-PARTITION – dela $3m$ tal i m tripplar med summan T var – är starkt NP-fullständigt (Garey och Johnson, 1975).
- **Skiss:** lappar $(m+1) \cdot a_i$ breda och 1 höga; täcke $(m+1) \cdot T$ brett och m högt. Lapparna är för långa för att stå på högkant, så varje rad måste fyllas av lappar med summan T.
- Lapptäcke är alltså starkt NP-fullständigt. Inget kappsäcksknep väntar runt hörnet.

$m = 2$: talen 6, 7, 7, 6, 6, 8
i två tripplar, en per rad

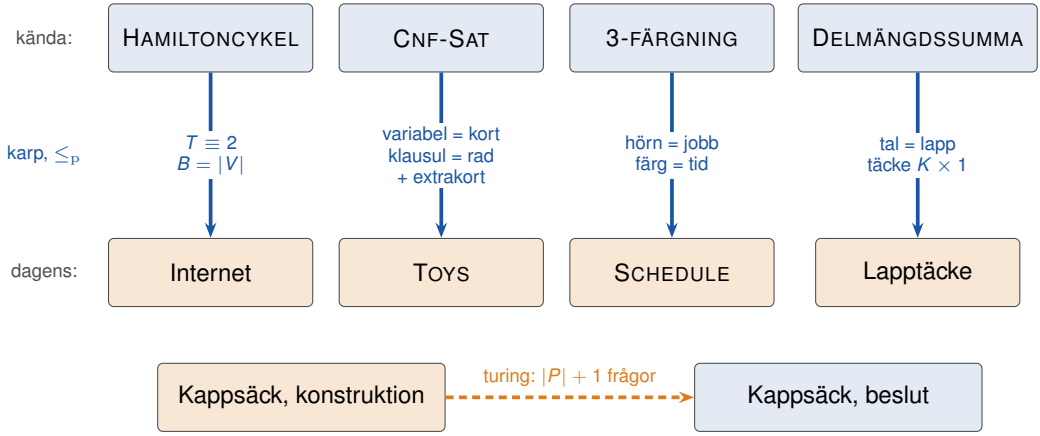


(alla bredder ska dessutom multipliceras med $m + 1 = 3$)

- **Måste koordinaterna vara heltal?** Ja, och det sker av sig självt. Längs en vågrät linje genom täcket ligger lapparnas kanter på delsummor av heltal. Formuleringen kan därför kräva heltal utan att tappa några lösningar.
- **Vad betyder ”vrida”?** Vi tolkar det som ett kvarts varv: bredd och höjd byter plats. Det är ett modelleringsval, och det är så facit gör.
- **Parkontrollen tar $O(n^2)$. Går det fortare?** Ja, $O(n \log n)$ med svepteknik. För ett NP-bevis är det överkurs: polynomiskt räcker, och ingen delar ut stilpoäng.
- **Kan man konstruera täcket med hjälp av beslutsproblemet, som i uppgift 1?** Ja. Att slänga överflödiga lappar ger vilka lappar som ska användas, och den allmänna satsen från uppgift 1 lovar att också placeringen går att få fram med polynomiskt många frågor.

Sammanfattning

Dagens reduktioner i en bild



Fyra NP-fullständighetsbevis efter samma recept, och en turingreduktion. Det enda som varierade var vilken kusin vi valde.

Var går gränsen mellan lätt och svårt?

	Lätt (polynomisk tid)	**Svårt** (NP-fullständigt)
Tillförlitligt nät	$T \equiv 1$: spännande träd	$T \equiv 2$: hamiltoncykel
Schemaläggning	$k = 2$: tvåfärgning med BFS	$k \geq 3$: trefärgning
Satisfierbarhet	högst två literaler per klausul	tre literaler per klausul
Kappsäck	små tal: tabell med $\lvert P \rvert \cdot S$ rutor	stora tal

- Skillnaden mellan lätt och svårt är förvånansvärt ofta skillnaden mellan två och tre.
- Sista raden är av ett annat slag. Där går gränsen inte i problemets struktur utan i hur stora tal indata får innehålla. Det är därför pseudopolynomiska algoritmer finns, och därför de inte motsäger något.
- För lapptäcke finns ingen sådan nödutgång: det är svårt även med små tal.

Vanliga fel i NP-fullständighetsbevis

- **Fel håll.** Det *kända* problemet ska lösas med hjälp av ditt, inte tvärtom.
- **Glömt NP.** NP-svårt är inte NP-fullständigt förrän problemet också ligger i NP.
- **Bara halva korrektheten.** Ja ska bli ja *och* nej ska bli nej. Den andra halvan visas som ”ja i den nya instansen ger ja i den gamla”.
- **Reduktionen tjuvkikar.** ”Välj hamiltoncykeln och . . . ” – reduktionen känner inte lösningen. Den översätter bara frågan.
- **Pseudopolynomiskt.** Slingor över talvärden och matriser med $B \cdot H$ rutor är exponentiella i indatas längd.
- **För stort certifikat.** Certifikatet ska vara polynomiskt långt, inte bara gå att kontrollera.
- **Turing i stället för Karp.** Flera anrop eller ett negerat svar duger inte i ett NP-svårighetsbevis.
- **Fel specialfall.** Reduktionen måste klara *varje* instans av det kända problemet. Däremot får den gärna bara skapa mycket speciella instanser av ditt: ringar, enmillimeterstäcken, korthögar med extrakort.

Nästa gång

- **Övning 11: approximationsalgoritmer.**
- Vad gör man när problemet är NP-svårt men ändå måste lösas före fredag?
- Vi har redan tjuvstartat: faktor-2-algoritmen för nätverksdesign i uppgift 2 är en sådan.

Mina slides finns på GitHub.

Bilderna bygger på

- Viggo Kanns uppgifter och lösningar till övning 10 (2026),
- övningsbilder av Anton Grensjö (2017), Lisa Li (2019) och Alice Heavey (2019).

Alla exempel och reduktioner är kontrollräknade med ett program, `verify.py`.